MapReduce and the New Software Stack

Presented by:
Dere Roshidat Oluwabukola
246515

MapReduce and the New Software Stack

- Big data Analysis is the term used for referring to modern data-mining which focused on the quick management of large amount of data.
- Software stack was invented to solve the problem of parallelism in many applications such as PageRank, Searches e.t.c.
- Distributed file system is a new form of file system used in software stack, it comprises of lager unit features than the disk blocks in other conventional operating systems. It also provide duplication and redundancy to protect against media failure when distributing data over thousands of low cost compute nodes.
- MapReduce is a higher-level programming system built after software stack.
- The implementation of MapReduce is to enable efficient calculation of large-scale data on computing cluster as well as resisting hardware failure during computation.

Distributed File Systems

The emergence of large-scale web services bring about the shift in Computation from the specialized parallel machines to the installation with thousands of independent compute nodes using commodity hardware, which enable the reduction in cost.

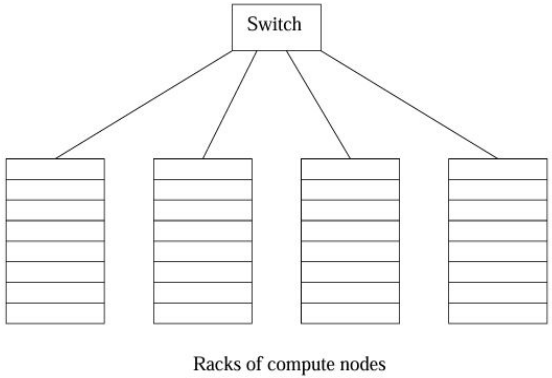
A new generation of programming system arises from the new computing facilities, which take advantage of parallelism and avoid the problem of failure when computing thousands of independent components on hardware.

Types of distributed file systems are:

- The Google File System (GFS)
- Hadoop Distributed File System (HDFS)
- Colossus

Physical Organization of Compute Nodes

Cluster computing is the new parallel-computing architecture where the computing nodes are stacked together on a rack and connected by networks such as gigabit Ethernet, the racks are then connected by another network or switch. The bandwidth of intra-rack Ethernet is not as strong as the interrack communication.



Physical Organization of Compute Nodes

The more the component of a system, the more frequent the failure at any given time. The following solutions can be carried out to solve the problems of nodes failure:

1. The files must be redundant, there is a tendency of losing the files when there is a disk crash and if not duplicated at different compute nodes.

2. The computations must be divided into tasks, so that it can be completed if one task fails.

Large-Scale File System Organization

Distributed file system (DFS) is the new system used for exploiting cluster computing. DFS can be used when large-scale data is involved. It can also be used when the files are rarely updated. It is not suitable for large data that is always updated.

Files are divided into chunks usually of 64 megabytes, each chunk is replicated 3 times and saved in three different nodes stacked on different ranks to avoid the problem of rack failure. The size of the chunk and the degree of replication has to be decided by the user.

Master node or name node is used for finding the chunk of a file, it is also replicated and a file system directory knows where to find each copy, the directory is also duplicated and can be found using DFS.

MapReduce

Several systems have implemented MapReduce as their computing style. It can be used for managing several large-scale, parallel computations and also resist hardware faults using just two functions such as Map and Reduce.

1. Chunks from the distributed file system are given to several Map tasks; it transforms the chunks into a sequence of key-value pairs. The way the key-value pair is produced depends on the user's code Map functions.

2. Master controllers collate all key-value pairs from each Map task and sort by key, which are then distributed across the Reduce tasks, ensuring all key-value pairs with the same keys are saved to the same Reduce task.

3. The Reduce tasks process a key at a time, adding up all the values associated, the process of combining is decided by the user’s code for the Reduce function.

Grouping by Key

- Key-value pairs are grouped by key with the values for each key forming a single list. The master controller process knows the number of Reduce tasks (r),uses a hash function to assign a key as argument and produces a bucket number from 0 to r −1.

- The master controller merges files from each Map task that are corresponding to a particular Reduce task and feeds the merge file as a sequence of key-value pairs to the system.

The Reduce Tasks

- The Reduce function takes a key and a list of values associated with that key as input1. This process is also referred to as a reducer .
- The Reduce function can output zero or more key-value pairs.
- The types of the key-value pairs output by the Reduce function are not required to match the input type. However, the output and input type are often the same.
- The Reduce task is responsible for executing one or more reducers.
- The outputs of all the Reduce tasks are finally merged into one file

The Map Task

The input to Map tasks are elements which can either be tuples or documents. Map function takes input elements and generates zero or more key-value pairs. The keys do not have to be unique, map tasks can produce multiple pairs of keys even from the same element.

Combiners

When a Reduce function is associative and commutative like addition, the order of combining values doesn’t affect the result. In such cases, some reducer’s work can be done during the Map phase.

Details of mapReduce Execution

- A MapReduce program is executed through a series of processes, tasks and file interactions.

- The user's program utilizes a MapReduce library like Hadoop to initiate a Master controller process and several Worker processes on different compute nodes. Workers typically handle either Map tasks or Reduce tasks, but not both.

- The Master controller is responsible for creating and assigning Map and Reduce tasks, typically with one Map task per input data chunk and fewer Reduce tasks to avoid too many intermediate files.